Report Helix_Intern_Candidates

Fialkovska Olya

Report Helix_Intern_Candidates....1
Executive Summary.... 3
Firmware Analysis Report.... 4
1. Basic Information....4
2. Initial File Type Analysis.... 4
3. Firmware Structure Analysis (Binwalk)....4
4. Extraction Attempt.... 5
5. Extracted Components....5
6. Initramfs Filesystem Structure....6
6.1 Filesystem Layout....6
6.2 Key Observations.... 6
6.3 Initramfs Boot Environment Analysis....7
6.4 Boot Script Analysis....7
7. Raw Firmware Fingerprint Discovery (Initramfs Layer).... 8
7.1 Version Metadata....8
7.2 Kernel Version.... 8
7.3 Architecture.... 8
7.4 Summary of Discovered Attributes (16.12.07)....9
7.5 Transition to Full Filesystem Analysis....9
8. Full Root Filesystem Extraction (Linux Environment)....9
8.1 Linux Environment Preparation.... 9
8.2 Mounting Host Firmware Repository in Linux VM.... 10
8.3 Full Firmware Extraction (16.12.07).... 10
8.4 SquashFS Content Structure Analysis.... 11
8.5 Active Package Configuration Analysis.... 11
8.6 Full Firmware Extraction (16.12.06).... 12
8.7 SquashFS Extraction (16.12.06).... 12
8.8 Firmware Delta Analysis (16.12.06 vs 16.12.07)....12
9. Extended Static Analysis.... 13
9.1.1 Strings-based Network Indicator Analysis (Version 16.12.06)....13
9.2.1 Network Primitive Indicators (Version 16.12.06)....14
9.1.2 Strings-based Network Indicator Analysis (Version 16.12.07)....14
9.2.2 Network Primitive Indicators (Version 16.12.07)....15
9.3 Cross-Version Static Comparison (16.12.06 vs 16.12.07)....15
9.4 Package File Type Identification (16.12.06 vs 16.12.07)....15
9.5 Secrets and Credential Artifact Scan....16
9.6 Network Artifact Scan (URLs + IP Indicators).... 16

9.7 Web Interface Discovery (webui)....17
9.8 Secure Remote Access Services (SSH).... 17
10. Custom Go Parser (Signature Scanner).... 18
10.1 Goal....18
10.2 Approach.... 18
10.3 Output Artifacts....18
10.4 Structural Conclusion and Limitation.... 19
11. Embedded Payload Carving (cat9k-rpbase.16.12.07)....19
12. Automated Multi-Layer Payload Extraction via Custom Go Parser....19
12.1 Compression Header Anomaly Analysis (GZIP Method 236).... 20
13. Structured Firmware Fingerprint Framework....21
13.1 Core Static Firmware Identifiers.... 21
13.2 Cross-Version Fingerprint Delta Analysis....22
14. Binary-Level Reverse Engineering Analysis (ELF Triage).... 23
15. Industrial-Scale Firmware Identification Implications.... 24
16. .pkg Container Deep Structure Analysis.... 25
16.1 Header Inspection via Hex Analysis....25
16.2 TLV Metadata Structure Hypothesis....25
16.3 TLV Metadata Discovery via Strings....25
16.4 Entropy Analysis of .pkg Container.... 26
16.5 Payload Offset Alignment Analysis....26
16.6 Modular Firmware Architecture (SquashFS Package Layer Recap)....26
16.7 Automated Package Signature Analysis (Custom Go Scanner Output)....27
16.8 Binary Comparison Between Firmware Builds (rpbase)....27
16.9 Embedded Binary Components (ELF Indicators)....27
17. Final Conclusion....28

Executive Summary

This report presents the results of a technical analysis of two Cisco IOS-XE firmware images:

C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.06.SPA.bin
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

The goal of this work was to investigate the internal structure of the firmware, extract embedded components, compare different firmware builds, and understand the packaging format used inside Cisco IOS-XE systems.

The analysis was performed using a combination of standard firmware analysis tools and custom tooling, including binwalk, strings, grep, unsquashfs, hex-level inspection, and a custom Go-based signature scanner developed specifically for this task.

Initial analysis revealed that the firmware follows a multi-layer architecture, consisting of:

- compressed data segments
- a Linux initramfs boot environment
- a SquashFS filesystem containing modular IOS-XE packages
- proprietary .pkg containers used to distribute runtime components

Because full filesystem extraction was not possible in the macOS environment, a Linux virtual machine was deployed to perform complete SquashFS extraction and deeper structural analysis.

Both firmware versions were successfully extracted and compared. The analysis shows that version 16.12.07 is a maintenance update, with the same architecture, kernel version, and package structure as 16.12.06.

Further inspection of the .pkg containers indicates that they use a proprietary modular packaging format, combining TLV-style metadata headers with multiple compressed payload segments.

To better understand this structure, a custom Go-based parser was implemented to detect embedded compression signatures and automatically carve potential payload fragments from the container.

Overall, the analysis demonstrates that Cisco IOS-XE firmware uses a modular, layered architecture, and it establishes a practical methodology for decomposing proprietary firmware containers and extracting reliable structural fingerprints across firmware versions.

Firmware Analysis Report

1. Basic Information

1.1 Target Files
Two firmware images were provided for analysis:
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.06.SPA.bin
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

1.2 File Integrity (SHA256)
SHA256 hash of the analyzed firmware:
7461e82266335646a6770a10b102fb8b895aeae88af82db3079e7ab82ce0cbb9
This hash uniquely identifies the firmware image and ensures integrity during analysis.

2. Initial File Type Analysis

Command used:
file C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

Output:
data

Interpretation:

The file utility was unable to detect a known standard file format.
This suggests that the firmware image uses a custom or complex binary structure.

3. Firmware Structure Analysis (Binwalk)

Command used:
binwalk C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

Findings
The binwalk scan identified several embedded components inside the firmware image.
GZIP compressed segment
Offset: 0x487B (18555)
Indicates the presence of compressed payload data embedded within the firmware container.

Initramfs archive
Offset: 0x4E9A20 (5,151,264)
File: initramfs.x86_64.cat9k.ramfs.cpio
Format: CPIO archive
Architecture: x86_64
This archive represents the boot initialization filesystem used during early system startup.

SquashFS filesystem
Version: 4.0
Compression: XZ
Image size: ~782 MB
This filesystem contains the primary runtime components of the IOS-XE system.

Structural Interpretation
The firmware image follows a multi-layer architecture consisting of:
compressed payload segments (GZIP)
a boot initialization environment (initramfs)
a compressed runtime filesystem (SquashFS)

This structure indicates a Linux-based firmware environment, where initramfs prepares the system during boot and SquashFS stores the modular IOS-XE runtime packages.

4. Extraction Attempt

Command used:
binwalk -e C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

Results:
GZIP segments were successfully extracted.
SquashFS filesystem extraction failed due to missing external extractor (sasquatch) on macOS.

Technical Note:
Automatic extraction of SquashFS requires additional tools not available in the current environment.
Further analysis will require a Linux-based environment for full filesystem extraction.

Resolution: This limitation was resolved by deploying a Linux VM with squashfs-tools for full extraction.

5. Extracted Components

Following the initial binwalk -e extraction, the firmware image produced the following embedded components:

Offset 0x487B → GZIP compressed data
Offset 0x4E9A20 → Initramfs archive (CPIO format)

These artifacts correspond to the compressed segments identified during the structural scan in Section 3 and confirm the presence of multiple embedded payload layers within the firmware image.

The initramfs component extracted from offset 0x4E9A20 was saved as:
decompressed.bin

6. Initramfs Filesystem Structure

To determine the nature of the extracted initramfs component, the following command was executed:
file decompressed.bin

Output:
ASCII cpio archive (SVR4 with no CRC)

This confirms that the extracted payload is a valid CPIO archive representing an initramfs filesystem for x86_64 architecture.

6.1 Filesystem Layout

After extracting the CPIO archive, a Linux-style directory structure was observed, including:

```
bin/
etc/
init
lib/
lib64/
usr/
sbin/
dev/
sys/
...
```

This layout is consistent with a minimal Linux boot environment.

6.2 Key Observations

The presence of /init indicates the boot initialization entry point.

Directories such as /bin, /sbin, /usr, /lib, and /lib64 confirm the presence of a standard Linux runtime environment.
Scripts such as rommon_to_env indicate Cisco-specific boot logic and environment handling.

Interpretation:
The extracted initramfs represents a Linux-based boot layer embedded within the firmware image. This confirms that the Cisco IOS-XE firmware operates on top of a Linux kernel and utilizes a conventional initramfs mechanism during system startup.

6.3 Initramfs Boot Environment Analysis

Observation: initramfs contains a minimal Linux runtime used before the main IOS-XE packages are loaded.

Typical directories present:

bin/, dev/, etc/, lib/, lib64/, sbin/, usr/, sys/

Cisco-specific boot scripts identified:
mount_packages.sh
verify_packages.sh
rommon_vars.sh

Interpretation: the initramfs layer prepares the environment and loads/verifies modular IOS-XE runtime packages during early boot.

6.4 Boot Script Analysis

verify_packages.sh

Performs digital signature verification:
verify_pkg --bootverify <package>

On success reports:

Package <pkg_path> is Digitally Signed

mount_packages.sh

Prepares and mounts runtime packages.

Uses rsync synchronization and invokes install engine:
install_engine.sh --operation install_fp_cc_boot
rommon_vars.sh

Loads ROMMON environment variables and configures system parameters before IOS-XE runtime startup.

Interpretation: boot flow is modular and integrity-checked; package authenticity is validated during initialization.

7. Raw Firmware Fingerprint Discovery (Initramfs Layer)

During recursive string inspection of the extracted initramfs archive, multiple version-identifying artifacts were discovered.

7.1 Version Metadata

The following firmware build identifiers were extracted:
RELVER=16.12.07
Build: 16.12.07
Version: 16.12.07.0.6565.1643829996
Codename: Gibraltar

These values uniquely identify the analyzed firmware image and represent stable build-level identifiers embedded within the boot layer.

7.2 Kernel Version

Inspection of the extracted filesystem revealed multiple kernel modules under:
lib/modules/4.4.202/

This confirms that the firmware operates on:
Linux kernel version 4.4.202

7.3 Architecture

The initramfs archive name:
initramfs.x86_64

confirms:
Architecture: x86_64

7.4 Summary of Discovered Attributes (16.12.07)

The following raw fingerprint artifacts were identified at the initramfs layer:
Firmware Version: 16.12.07
Full Build String: 16.12.07.0.6565.1643829996
Codename: Gibraltar
Linux Kernel: 4.4.202
Architecture: x86_64

These attributes were extracted directly from the boot environment and represent primary build-level identifiers.

7.5 Transition to Full Filesystem Analysis

Although reliable version metadata was identified at the initramfs layer, the primary SquashFS root filesystem remained unextracted in the macOS environment due to missing external extraction utilities.

This limitation was subsequently resolved by deploying a Linux virtual environment (Section 8), enabling deeper structural and package-level analysis.

8. Full Root Filesystem Extraction (Linux Environment)

Since SquashFS extraction failed in macOS due to missing external extractors (e.g., sasquatch), a Linux virtual environment was deployed using UTM on Apple Silicon. Ubuntu ARM64 was installed to provide native support for SquashFS extraction tools.

8.1 Linux Environment Preparation

To enable full extraction of the SquashFS filesystem, a Linux virtual machine was deployed using UTM on Apple Silicon.
Ubuntu 22.04 ARM64 was installed as the guest operating system.

The following analysis tools were installed:
sudo apt update
sudo apt install binwalk squashfs-tools

The binwalk utility was used for firmware structure analysis and automated extraction, while squashfs-tools provides native support for extracting SquashFS filesystems via unsquashfs.

This setup enables complete root filesystem extraction not supported in the macOS environment.

8.2 Mounting Host Firmware Repository in Linux VM

To enable full filesystem extraction, the host firmware repository was mounted inside the Linux virtual machine using VirtFS (9P protocol).

Commands used:
sudo mkdir /mnt/share
sudo mount -t 9p -o trans=virtio,version=9p2000.L share /mnt/share
ls /mnt/share

Result:
The following firmware images were successfully detected inside the Linux environment:
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.06.SPA.bin
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin

This confirms proper host-to-VM integration and readiness for full filesystem extraction.

8.3 Full Firmware Extraction (16.12.07)

To perform full extraction of the root filesystem, the firmware image C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin was processed inside a Linux virtual machine (UTM).

The firmware file was accessed via a shared folder mounted at /mnt/share. Since this mount was used only as a source location, all extracted data was written to the user's home directory inside the Linux environment.

An initial manual carving approach based on the identified SquashFS offset was considered. However, due to performance limitations associated with byte-level extraction of large firmware segments, the extraction strategy was revised to use automated recursive extraction via binwalk.

The following commands were used:
mkdir -p ~/extraction_07

```
cd ~/extraction_07
binwalk -Me /mnt/share/C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.07.SPA.bin
```

This method enabled reliable extraction of embedded components, including the SquashFS root filesystem, using native Linux-based tools.

8.4 SquashFS Content Structure Analysis

The extracted SquashFS filesystem does not contain a standard Linux root directory layout. Instead, it contains modular IOS-XE package components (.pkg files) and a packages configuration file.

Observed structure includes:
cat9k-rpbase.16.12.07.SPA.pkg
cat9k-webui.16.12.07.SPA.pkg
cat9k-sipbase.16.12.07.SPA.pkg
…
packages.conf

This indicates that the firmware utilizes a modular package-based architecture typical for Cisco IOS-XE systems.
The SquashFS layer acts as a container for IOS-XE software packages rather than a traditional root filesystem.

8.5 Active Package Configuration Analysis

The packages.conf file was examined to determine active IOS-XE software components.

The configuration confirms that the firmware operates in modular package mode, where multiple functional components are loaded independently.

Key observed packages include:
cat9k-rpbase.16.12.07.SPA.pkg
cat9k-webui.16.12.07.SPA.pkg
cat9k-sipbase.16.12.07.SPA.pkg
cat9k-srdriver.16.12.07.SPA.pkg
cat9k-guestshell.16.12.07.SPA.pkg

This confirms that the firmware utilizes a modular architecture typical of Cisco IOS-XE systems and that Web UI, routing services, and guest shell functionality are enabled.

The package naming convention further confirms firmware version 16.12.07.

8.6 Full Firmware Extraction (16.12.06)

To perform comparative analysis, the firmware image
C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.06.SPA.bin
was extracted using the same Linux-based workflow.

The extraction was performed inside the virtual machine environment using recursive binwalk processing.

Commands used:
mkdir -p ~/extraction_06
cd ~/extraction_06
binwalk -Me /mnt/share/C9200_9300_9400_9500_9600_cat9k_iosxe.16.12.06.SPA.bin

This ensured methodological consistency between firmware builds 16.12.06 and 16.12.07, allowing accurate structural and package-level comparison.

8.7 SquashFS Extraction (16.12.06)

The firmware image version 16.12.06 was extracted using the same Linux-based workflow applied to version 16.12.07.

The SquashFS filesystem was successfully unpacked, revealing the same modular IOS-XE package-based architecture.

Observed structure includes:
cat9k-rpbase.16.12.06.SPA.pkg
cat9k-webui.16.12.06.SPA.pkg
cat9k-sipbase.16.12.06.SPA.pkg
packages.conf

This confirms structural consistency between firmware builds 16.12.06 and 16.12.07.

8.8 Firmware Delta Analysis (16.12.06 vs 16.12.07)

A direct comparison of the extracted packages.conf files from firmware versions 16.12.06 and 16.12.07 was performed using the diff utility.

The comparison revealed:
Identical package structure
Identical module set
No additional or removed components

Only version identifiers updated from 16.12.06 to 16.12.07

All functional components (rpbase, webui, sipbase, srdriver, guestshell, wlc, etc.) remain consistent between both builds.

This indicates that version 16.12.07 represents a maintenance or patch-level update rather than a structural or architectural modification.

The firmware architecture, modular packaging scheme, and component composition remain unchanged across both analyzed versions.

9. Extended Static Analysis

9.1.1 Strings-based Network Indicator Analysis (Version 16.12.06)

To identify potential network-related functionality within the firmware packages, a controlled keyword-based strings inspection was performed on the primary runtime package:

Command used:
strings -a -n 6 cat9k-rpbase.16.12.06.SPA.pkg | grep -i -m 30 "ssh"

Results:
Multiple references to SSH-related components were identified, including:

cli_ssh.beam
confd_ssh.beam
netconf_ssh
netconf_server
netconf_tcp

Interpretation:
The presence of SSH-related Erlang modules (.beam files) indicates that the firmware includes embedded SSH service functionality.
Additionally, references to NETCONF over SSH suggest support for secure remote management via standardized network configuration protocols.

9.2.1 Network Primitive Indicators (Version 16.12.06)

To further inspect network surface exposure, recursive keyword inspection was performed across all package files:

Command used:
strings *.pkg | grep -i -E -m 30 "socket|bind|connect|ssl"

Results:
The following network-related indicators were observed:
SSL
ssl
SSLe
references consistent with cryptographic / secure transport usage

Interpretation:
The presence of SSL-related strings confirms that encrypted communication mechanisms are embedded within the firmware packages.

Combined with previously identified SSH references, this suggests:
Support for encrypted management sessions
TLS/SSL-based secure transport
Network socket-level operations within core components

No abnormal or unexpected networking keywords were observed during initial static triage.

9.1.2 Strings-based Network Indicator Analysis (Version 16.12.07)

To perform static triage of network-related functionality, keyword-based string inspection was conducted on the primary runtime package:

Command used:
strings -a -n 6 cat9k-rpbase.16.12.07.SPA.pkg | grep -i -m 30 "ssh"
Observed Indicators

The following SSH-related artifacts were identified:
cli_ssh.beam
confd_ssh.beam
netconf_ssh
netconf_server
netconf_tcp

These findings are consistent with embedded SSH service support and NETCONF-over-SSH remote management functionality.

9.2.2 Network Primitive Indicators (Version 16.12.07)

To further evaluate low-level networking primitives, the following command was executed:
strings *.pkg | grep -i -E -m 30 "socket|bind|connect|ssl"

Observed Network/Encryption Indicators
SSL
ssl
variations of SSL-related references
encrypted communication artifacts

This confirms the presence of cryptographic transport mechanisms and secure communication capabilities embedded within the firmware.
No unexpected or anomalous network-related indicators were detected during static inspection.

9.3 Cross-Version Static Comparison (16.12.06 vs 16.12.07)

A direct comparison of string-based network indicators between firmware versions 16.12.06 and 16.12.07 was performed.

Comparison Findings

Both versions exhibit:
Presence of SSH-related Erlang modules (.beam files)
NETCONF over SSH functionality
SSL-based encrypted transport indicators

Similar distribution of network-related strings

No structural differences in SSH modules, SSL indicators, or networking primitives were observed between the two firmware versions.

9.4 Package File Type Identification (16.12.06 vs 16.12.07)

To determine whether IOS-XE package files (.pkg) use a recognizable standard format, the file utility was executed against the primary runtime package in both firmware versions.

Commands used:
file cat9k-rpbase.16.12.06.SPA.pkg
file cat9k-rpbase.16.12.07.SPA.pkg

Output:
data

Interpretation:
The file utility did not detect a known file signature for the examined .pkg files in either version.
This suggests that Cisco IOS-XE .pkg components are not standard archives (e.g., tar/gzip) and are not directly identifiable executables via generic magic signatures.

Both versions (16.12.06 and 16.12.07) behaved identically under this check, indicating that the package container format remains consistent across the analyzed releases.

9.5 Secrets and Credential Artifact Scan

Goal: detect plaintext secrets and credentials in extracted layers.

Commands used:
grep -R "BEGIN RSA PRIVATE KEY" .
grep -R "BEGIN OPENSSH PRIVATE KEY" .
grep -R "BEGIN CERTIFICATE" .
grep -Ri "password" .
grep -Ri "token" .

Results:
Private keys: not found (no plaintext RSA/OpenSSH private keys).
Certificates: no BEGIN CERTIFICATE blocks found.
Credentials: no clear plaintext credentials found (password/token scan produced no direct plaintext hits).

Interpretation: no obvious plaintext secrets were discovered at the extracted filesystem layer.

9.6 Network Artifact Scan (URLs + IP Indicators)

Commands used:

```
grep -Ri "http" .
grep -E -R "[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+" .
```

Results:
http matches were detected inside binary payload fragments (compressed/compiled modules), not as plaintext config endpoints.

Examples observed in output include:
http://www.pygkt.org
http://www.freedesktop.org/standards/dbus

IPv4-like matches were detected inside binary artifacts and carved fragments, not in plaintext configuration files.

Interpretation:
No explicit hardcoded external endpoints or IPv4 addresses were identified in readable config files.

Detected strings most likely originate from compiled code / embedded libraries.

9.7 Web Interface Discovery (webui)

Command used:
```
grep -Ri webui .
```

Evidence examples:
/usr/binos/conf/webui.conf
/usr/binos/conf/webui-servers.conf
/usr/binos/conf/webui_upgrade_helper.sh
/usr/binos/conf/webui_smu_helper.sh
/lib/tmpfiles.d/webui.conf

references to rp_webui package mounting

Interpretation: firmware includes an integrated web management interface; nginx-related references suggest an embedded web server component.

9.8 Secure Remote Access Services (SSH)

Command used:
```
grep -Ri "ssh" .
```

Evidence examples:

/lib/systemd/system/sshd.service

/etc/services: ssh 22/tcp
/etc/services: ssh 22/udp
/etc/cron: sshd
/etc/avahi/services/ssh.service

Interpretation: SSH daemon is present, confirming secure remote administration support.

10. Custom Go Parser (Signature Scanner)

To further analyze the internal structure of Cisco IOS-XE .pkg containers, a custom Go-based signature scanner was developed.

10.1 Goal

The primary objective of the parser was to detect embedded payload segments within proprietary Cisco .pkg files.

Since standard utilities (file, binwalk) did not identify a recognizable archive format, a format-aware scanning approach was required to locate compressed or structured payload boundaries inside the container.

10.2 Approach

The parser was implemented in Go and designed to scan binary data for known magic byte signatures associated with common compression and filesystem formats.

The scanning process targeted signatures including:
GZIP (1f 8b 08)
XZ (fd 37 7a 58 5a 00)
SquashFS (hsqs)
ELF (7f 45 4c 46)

When a known signature was detected, the parser recorded the corresponding offset within the .pkg container for further manual or automated carving.

This approach enabled structural decomposition of the proprietary container without relying on predefined header specifications.

10.3 Output Artifacts

The parser generated:
Offset locations of detected payload candidates
Candidate segment boundaries for carving
Structured data suitable for further automated extraction workflows

These offsets were subsequently used in manual carving experiments (Section 11) and automated multi-layer extraction (Section 12).

10.4 Structural Conclusion and Limitation

The analysis indicates that Cisco IOS-XE .pkg files do not conform to standard archive formats (e.g., tar, gzip, or standalone SquashFS images).

Instead, the container appears to embed multiple compressed and structured payload segments within a proprietary modular format.

Because the full container header specification is not publicly documented, deeper reverse engineering would require formal header reconstruction and format specification derivation.

However, the signature-scanning approach provides a practical and scalable method for decomposing .pkg containers without prior format knowledge.

11. Embedded Payload Carving (cat9k-rpbase.16.12.07)

Using the custom Go signature scanner, an embedded SquashFS-related marker was identified at offset 0x1A742280.
Manual carving using dd confirmed that this region contains multiple XZ-compressed segments.

This suggests that Cisco .pkg files are proprietary containers embedding multiple compressed payload blocks rather than standard archive formats.

12. Automated Multi-Layer Payload Extraction via Custom Go Parser

The automated parser workflow was validated on 16.12.06 to confirm repeatability across builds and to increase fragment coverage.

Following the initial manual carving approach described in Section 11, a more advanced automated analysis workflow was implemented.

A custom Go-based parser (pkgscan2) was developed to:
Detect embedded payload boundaries using known magic byte signatures
Generate structured JSON reports
Automatically carve identified segments into separate binary artifacts
Compute SHA256 hashes for each carved artifact
Enable iterative post-processing analysis

The parser identified multiple embedded compression signatures inside cat9k-rpbase.16.12.06.SPA.pkg, including:
XZ compressed segments
GZIP compressed segments
SquashFS-related markers (hsqs)
Additional compression fragments distributed throughout the container

Automated carving produced hundreds of discrete binary fragments, ranging from small metadata blocks to larger compressed modules (200–400 KB).

Subsequent triage using:
binwalk carved/*.bin

confirmed that the .pkg container does not contain a single monolithic filesystem image. Instead, it embeds numerous individually compressed modules.

Structural Conclusion
The Cisco IOS-XE .pkg format appears to function as a proprietary modular container rather than a traditional archive format (e.g., tar, cpio, or squashfs-only image).

Key characteristics observed:
Multi-layer container structure
Distributed compressed artifacts (XZ and GZIP)
No single large payload block
Fragmented modular packaging

This indicates a componentized firmware architecture where runtime modules are independently compressed and bundled within a proprietary container structure.

Such architecture supports:
Modular loading of functional components
Fine-grained version control
Patch-level updates without structural reorganization

This confirms that IOS-XE .pkg files implement a layered, modular packaging design consistent with enterprise-grade network operating systems.

12.1 Compression Header Anomaly Analysis (GZIP Method 236)

During post-processing of carved GZIP fragments generated by the automated Go parser, additional compression header validation was performed.

Hex-level inspection (xxd) confirmed the presence of a standard GZIP magic signature:

1f 8b 08

However, attempts to decompress using standard tools (gunzip) failed.
The file utility reported:

gzip compressed data, reserved method 236, encrypted

The compression method identifier (236) does not correspond to the standard GZIP method (8 – DEFLATE).

This suggests that embedded GZIP-like segments inside the Cisco .pkg container:
Utilize non-standard compression method identifiers
May include proprietary header modifications
May embed encryption flags or extended metadata
Are not directly decompressible using standard userland tools

Technical Implication
The presence of GZIP magic bytes combined with a non-standard method value indicates that compression streams are wrapped or modified within the proprietary packaging format.

This reinforces the earlier conclusion that the Cisco IOS-XE .pkg container:
Does not store standalone archive objects
Embeds structured compression streams within a proprietary modular container
Requires format-aware parsing rather than generic archive extraction

13. Structured Firmware Fingerprint Framework

Section 7 documents raw fingerprint discovery for firmware version 16.12.07 at the initramfs layer.
This section formalizes those artifacts into a structured framework suitable for automated identification and cross-version comparison across firmware builds 16.12.06 and 16.12.07.

13.1 Core Static Firmware Identifiers

The following attributes were identified as stable fingerprintable indicators across both analyzed firmware builds (16.12.06 and 16.12.07):

Category	Attribute	16.12.06	16.12.07	Stability	Extraction Layer
Firmware Version	RELVER	16.12.06	16.12.07	High	initramfs
Full Build String	Version Identifier	16.12.06.*	16.12.07.0.6565.1643829996	High	initramfs / strings
Codename	Gibraltar	Gibraltar	High	initramfs	
Kernel Version	4.4.202	4.4.202	High	lib/modules	
Architecture	x86_64	x86_64	High	initramfs filename	

Notes:
The full build string for 16.12.07 was extracted directly from the initramfs layer.

The 16.12.06 build identifier follows the same structural pattern, with version-level differences corresponding to the release update.

All core architectural attributes remain stable across both firmware builds.

These identifiers are deterministic and suitable for automated firmware classification at scale.

13.2 Cross-Version Fingerprint Delta Analysis

A structured comparison between firmware versions 16.12.06 and 16.12.07 was conducted to evaluate fingerprint-level differences.

Observed differences:
Firmware version string updated
Full build identifier updated
Package filename suffix updated

No observed differences in:
Kernel version
Modular packaging architecture
Active package set
SSH/NETCONF functionality
SSL-related transport indicators

Interpretation:
Firmware version 16.12.07 represents a maintenance or patch-level update with no structural modifications to packaging, kernel, or networking components.

From a fingerprinting perspective, version differentiation relies primarily on:
RELVER variable
Full build string
Package filename suffix

All structural and architectural indicators remain stable between builds.

14. Binary-Level Reverse Engineering Analysis (ELF Triage)

During ELF triage of the extracted initramfs filesystem, a statically linked ELF executable (487B) was identified and analyzed.

Header inspection using readelf -h revealed:
ELF Class: 64-bit (ELF64)
Endianness: Little endian
OS/ABI: UNIX – System V
File Type: EXEC (Executable file)
Architecture: x86-64 (Advanced Micro Devices X86-64)
Linking: Statically linked
Entry Point: 0x100000

Interpretation:

The presence of a statically linked x86-64 executable confirms that the firmware contains embedded userland utilities compiled specifically for the target platform. Static linking suggests portability and reduced runtime dependency on shared libraries, which is common in boot or embedded environments.

This validates that executable components exist at the compiled binary level and are not solely inferred from higher-level package metadata.

Network Functionality Indicators
String-level inspection of the statically linked ELF executable revealed multiple networking-related artifacts, including:
hub_port_connect
xs_local_setup_socket
xs_tcp_setup_socket
/var/run/rpcbind.sock
TRACE_SYSTEM_SS_CONNECTING
TRACE_SYSTEM_SS_CONNECTED
TRACE_SYSTEM_SS_DISCONNECTING

These findings indicate the presence of compiled networking logic, socket-level communication handling, and connection state management directly embedded within the executable.

This confirms that network interaction capabilities exist at the binary implementation layer and are not solely inferred from high-level package metadata or configuration artifacts.

15. Industrial-Scale Firmware Identification Implications

The conducted analysis demonstrates that Cisco IOS-XE firmware follows a layered and modular architecture enabling deterministic fingerprint extraction across multiple abstraction layers.

Fingerprintable attributes were identified at:
Boot layer (RELVER variables inside initramfs)
Kernel layer (Linux kernel 4.4.202)
Packaging layer (.pkg naming conventions)
Container layer (embedded compression signatures and proprietary wrapping)
Binary layer (ELF architecture and networking-related symbols)

This multi-layer structure allows reliable firmware fingerprinting even when individual artifacts are modified.

From an automation perspective, firmware identification at scale could be achieved by:
Parsing RELVER variables from initramfs
Detecting modular package naming conventions

Hashing carved ELF executables
Identifying SSH/NETCONF indicators
Detecting proprietary compression signatures inside .pkg containers

This layered approach increases resilience and enables high-confidence firmware classification across large firmware datasets.

16. .pkg Container Deep Structure Analysis

16.1 Heяder Inspection via Hex Analysis

Goal: identify whether .pkg begins with structured metadata.

Command used:
xxd -g 1 -l 512 cat9k-rpbase.16.12.06.SPA.pkg

Observed header fields (examples):
KEY_TLV_PACKAGE_COMPATIBILITY
FRU_RP_TYPE
KEY_TLV_PACKAGE_BOOTARCH
ARCH_i686_TYPE
TLV_BOARD_COMPAT
KEY_TLV_CRYPTO_KEYSTRING
CW_IMAGE
CW_VERSION
CW_FULL_VERSION

Interpretation: .pkg begins with an ASCII-rich metadata header consistent with a structured format.

16.2 TLV Metadata Structure Hypothesis

Observation: repeated KEY_TLV_* suggests Type–Length–Value (TLV)-style metadata encoding.

Likely encoded categories:
package compatibility parameters
board type constraints
architecture requirements
firmware version identifiers
cryptographic metadata

Implication: the loader can validate package compatibility before loading modules.

16.3 TLV Metadata Discovery via Strings

Observation: string inspection also shows TLV-like metadata keys:
KEY_TLV_PACKAGE_COMPATIBILITY
KEY_TLV_PACKAGE_BOOTARCH

KEY_TLV_BOARD_COMPAT
KEY_TLV_CRYPTO_KEYSTRING

Interpretation: this supports the TLV header hypothesis and reinforces that .pkg is not a simple archive.

16.4 Entropy Analysis of .pkg Container

Command used:
binwalk -E cat9k-rpbase.16.12.06.SPA.pkg

Result:
Rising entropy edge at 0x0 (entropy ≈ 0.999886)
Interpretation: most of the container is high-entropy data (compressed/encrypted-like payloads), with a smaller metadata region.
Layout summary (high-level):
[TLV metadata header]
[compressed firmware payloads]

16.5 Payload Offset Alignment Analysis

Example payload offset (from firmware analysis):
0x4E9A20

4KB alignment test:
0x4E9A20 mod 4096 = 2592

Interpretation: not aligned to common flash page boundaries → supports “structured container with internal offsets”, not a raw flash image.

16.6 Modular Firmware Architecture (SquashFS Package Layer Recap)

Observation: extracted SquashFS contains multiple IOS-XE packages instead of a monolithic root filesystem (see Section 8.4–8.5 for detailed extraction analysis).

Example modules:
cat9k-rpbase
cat9k-sipbase
cat9k-sipspa
cat9k-webui
cat9k-wlc
cat9k-cc_srdriver

Interpretation: IOS-XE firmware is modular; rpbase acts as core runtime, other packages provide platform subsystems.

16.7 Automated Package Signature Analysis (Custom Go Scanner Output)

Target package:
cat9k-rpbase.16.12.06.SPA.pkg

Size: 536,691,716 bytes (~512 MB)

Detected signatures (examples):
SquashFS: 0x404 (1028)
XZ: 0x464 (1124)
XZ: 0xAE60 (44640)
GZIP: 0xDD14 (56596)
XZ: 0x1A388 (107400)
GZIP: 0x1A669 (108137)
GZIP: 0x1D2A1 (119457)
GZIP: 0x20DD8 (134616)
XZ: 0x22FC8 (143304)

Interpretation:
.pkg stores multiple compressed payload segments, not one monolithic filesystem.
Mixed formats (SquashFS/XZ/GZIP) imply a multi-layer modular container.

16.8 Binary Comparison Between Firmware Builds (rpbase)

Command used:
cmp -l cat9k-rpbase.16.12.06.SPA.pkg cat9k-rpbase.16.12.07.SPA.pkg

Result:
files differ starting from the first byte (expected across separate builds)

Interpretation:
differences likely due to timestamps/metadata/compression artifacts/minor patches
consistent with prior conclusion: 16.12.07 is maintenance-level, not architectural redesign

16.9 Embedded Binary Components (ELF Indicators)

Observation: strings on rpbase reveals multiple ELF markers inside the container.

Interpretation: .pkg embeds multiple compiled executables/components (not a single executable). Together with compression signatures, this supports the view of .pkg as a modular distribution container.

17. Final Conclusion

This assessment demonstrates a full-spectrum firmware analysis workflow, progressing from initial file triage to binary-level reverse engineering.

Key achievements include:

Identification of a multi-layer firmware architecture (initramfs + SquashFS + proprietary .pkg containers)

Full cross-version structural comparison between 16.12.06 and 16.12.07

Development of a custom Go-based signature scanner for automated payload detection

Discovery of proprietary compression anomalies (GZIP method 236)

Structured firmware fingerprint framework suitable for automated identification

Binary-level validation of embedded networking functionality within statically linked ELF executables

The firmware exhibits a modular, componentized architecture optimized for maintainability and patch-level updates.

The analysis establishes a repeatable methodology for decomposing proprietary firmware formats and extracting reliable fingerprintable attributes across structural, packaging, and binary layers.